

PROJE6_RAPOR

BULUT BİLİŞİM (3522) — PROJE 6 RAPORU

Video Akışı ve Görsel İşleme Uygulaması

Öğrenci: Arife Ebrar Üstüner

Öğrenci No: 23291207

Ders: Bulut Bilişim (3522)

Tarih: Mayıs 2026

Proje Bağlantıları

- **GitHub Repository:** <https://github.com/Ebrar3/Bulut-proje/tree/main/FİNAL/PROJE-6>
- **Proje Sunum Videosu:** https://drive.google.com/drive/folders/1ye-eWImLOZt989SDqtbN0zsUojzg4wV?usp=drive_link

1. Projenin Amacı

Bu projede, AWS'nin yapay zeka tabanlı görüntü işleme servisi olan **Amazon Rekognition** kullanılarak fotoğraflar ve videolar üzerinde otomatik analiz yapan bir web uygulaması geliştirilmiştir. Projenin temel hedefleri:

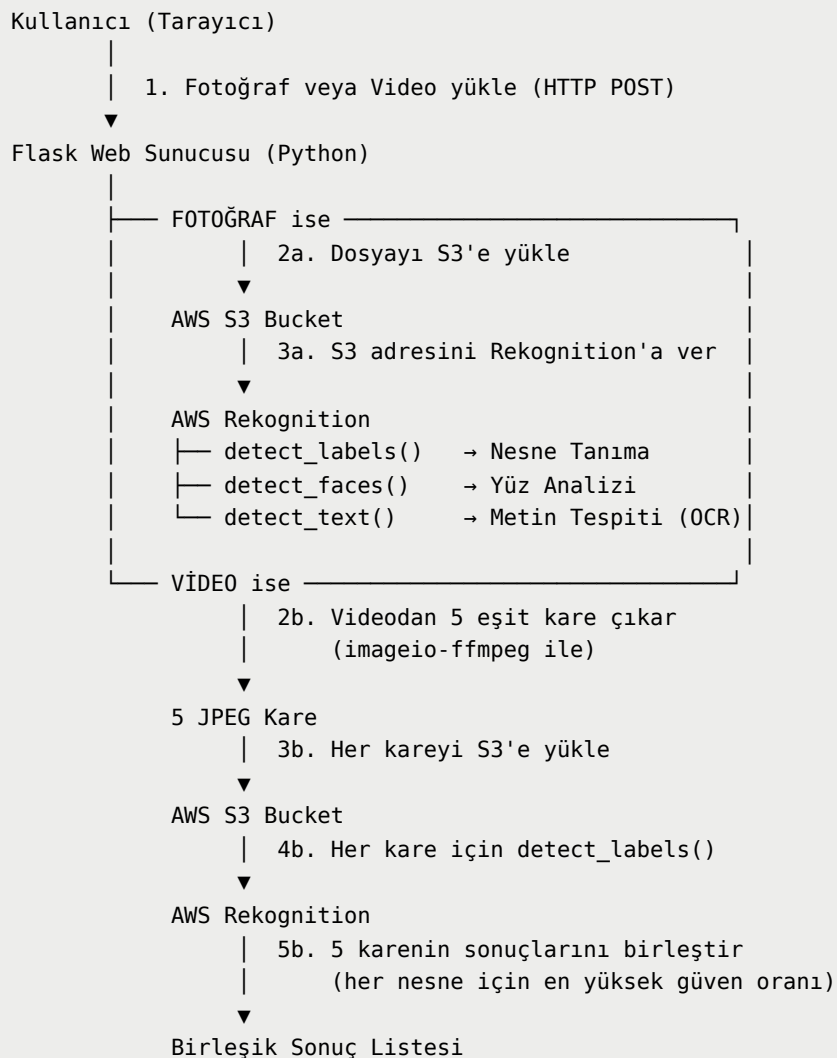
- Kullanıcının yüklediği fotoğraf veya videoda **nesne ve sahne tanıma** yapmak
- Fotoğrafta bulunan **insan yüzlerini analiz etmek** (yaş tahmini, duygu tespiti, aksesuar tespiti)
- Görseldeki **yazıları otomatik okumak** (OCR — Optik Karakter Tanıma)
- Videolar için **kare bölme yöntemiyle** bulut tabanlı video analizi gerçekleştirmek
- Tüm bu işlemleri **makine öğrenmesi eğitimi gerektirmeden**, AWS'nin hazır AI servislerini kullanarak yapmak

2. Kullanılan Teknolojiler ve Mimari

2.1 Teknoloji Yığını

Katman	Teknoloji	Açıklama
Backend	Python 3 + Flask	Web sunucusu ve istek yönetimi
Dosya Depolama	AWS S3	Yüklenen dosyaları bulutta saklar
Yapay Zeka	AWS Rekognition	Nesne, yüz, metin analizi
SDK	boto3	Python için AWS kütüphanesi
Video İşleme	imageio + imageio-ffmpeg	Videodan kare çıkarma
Arayüz	HTML + CSS (Dark Mode)	Modern kullanıcı arayüzü
Ortam Yönetimi	python-dotenv	Güvenli kimlik bilgisi yönetimi

2.2 Sistem Mimaris



4. Sonuçları kullanıcıya göster

Flask → result.html render → Tarayıcıda göster

2.3 Dosya Yapısı

```
PROJE-6/
├─ app.py                → Ana Flask uygulaması (routing, iş akışı)
├─ rekognition_service.py → AWS Rekognition API fonksiyonları
├─ s3_service.py         → AWS S3 yükleme fonksiyonları
├─ video_service.py      → Videodan kare çıkarma (imageio)
├─ templates/
│   ├─ index.html        → Dosya yükleme sayfası (drag & drop)
│   └─ result.html       → Analiz sonuçları sayfası
├─ requirements.txt      → Python bağımlılıkları
├─ .env                 → AWS kimlik bilgileri (gizli, GitHub'a atılmaz)
├─ .gitignore           → .env ve cache dosyalarını korur
└─ docs/
    └─ PROJE6_RAPOR.md   → Bu rapor
```

3. AWS Servisleri Kurulumu

3.1 Amazon S3 Bucket Oluşturma

Yüklenen dosyaları depolamak için S3 bucket oluşturuldu:

Ayar	Değer
Bucket Adı	proje6-rekognition-320042237584
Bölge	eu-central-1 (Frankfurt)
Erişim	Public read (Bucket Policy ile)

Bucket Policy — Herkese okuma izni:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::proje6-rekognition-320042237584/*"
  }]
}
```

Bu policy, uygulama sonuç sayfasında görselin önizlemesinin tarayıcıda görünebilmesi için gereklidir. Olmadan S3'teki görseller "Access Denied" hatası verir.

3.2 IAM Kullanıcı İzinleri

Flask uygulamasının AWS servislerine erişebilmesi için **ebrar** IAM kullanıcısına şu politikalar eklendi:

Politika	Açıklama
✓ AmazonS3FullAccess	S3'e dosya yükleme/silme
✓ AmazonRekognitionFullAccess	Rekognition API çağrıları

3.3 Ortam Değişkenleri (.env)

```
AWS_ACCESS_KEY_ID=AKIAUVBAA72ICWFN7LOC
AWS_SECRET_ACCESS_KEY=*****
AWS_REGION=eu-central-1
S3_BUCKET_NAME=proje6-rekognition-320042237584
```

⚠ **.env** dosyası **.gitignore** 'a eklendiğinden GitHub'a yüklenmez ve güvencedir.

4. Uygulama Kodunun Açıklaması

4.1 **app.py** — Ana Flask Sunucusu

Tüm iş akışını yönetir. Kullanıcıdan dosya alır, S3'e yükler, Rekognition ile analiz eder ve sonuçları HTML şablonuna render eder.

Fotoğraf için akış:

```
@app.route("/analyze", methods=["POST"])
def analyze():
    file = request.files["file"]
    s3_key, s3_url = upload_to_s3(file, unique_filename)
    bucket_name = os.getenv("S3_BUCKET_NAME")

    results["labels"] = detect_labels(bucket_name, s3_key) # Nesne tanıma
    results["faces"] = detect_faces(bucket_name, s3_key) # Yüz analizi
    results["text"] = detect_text(bucket_name, s3_key) # OCR
    return render_template("result.html", results=results)
```

Video için akış (kare birleştirme):

```
frames = extract_frames(file, num_frames=5)
merged_labels = {}

for i, frame_bytes in enumerate(frames, 1):
    frame_key, _ = upload_bytes_to_s3(frame_bytes, f"frames/frame{i}.jpg")
    frame_labels = detect_labels(bucket_name, frame_key, max_labels=10)

    # Her nesne için en yüksek güven oranını sakla
    for lbl in frame_labels:
        name = lbl["name"]
        if name not in merged_labels or lbl["confidence"] > merged_labels[name]["confidence"]:
            merged_labels[name] = lbl

# Güven oranına göre sıralı birleşik sonuç
results["labels"] = sorted(merged_labels.values(),
                           key=lambda x: x["confidence"],
                           reverse=True)[:15]
```

4.2 rekognition_service.py — Yapay Zeka İşlemleri

AWS Rekognition API'siyle iletişim kuran servis modülüdür. Üç temel fonksiyon içerir:

`detect_labels()` — Nesne Tanıma:

```
response = rekognition_client.detect_labels(
    Image={"S3Object": {"Bucket": bucket_name, "Name": s3_key}},
    MaxLabels=15,
    MinConfidence=70
)
# Döndürülen sonuç: [{"name": "Person", "confidence": 99.5, "emoji": "👤"}, ...]
```

`detect_faces()` — Yüz Analizi:

```
response = rekognition_client.detect_faces(
    Image={"S3Object": {"Bucket": bucket_name, "Name": s3_key}},
    Attributes=["ALL"] # Yaş, duygu, gözlük, sakal vb. tüm özellikler
)
```

`detect_text()` — Metin Tespiti (OCR):

```
response = rekognition_client.detect_text(
    Image={"S3Object": {"Bucket": bucket_name, "Name": s3_key}}
```

)

4.3 s3_service.py — Bulut Depolama

Dosya ve kare verilerini S3'e yükler:

```
# Fotoğraf yükleme
s3_client.upload_fileobj(file_obj, BUCKET_NAME, f"uploads/{filename}")

# Video karesi yükleme (bytes olarak)
s3_client.upload_fileobj(io.BytesIO(data_bytes), BUCKET_NAME, frame_key)
```

4.4 video_service.py — Video Kare Çıkarma

Codec bağımsız çalışan video işleme modülüdür. `imageio-ffmpeg` kütüphanesi kullanılarak telefon videoları dahil tüm yaygın formatlar desteklenmektedir:

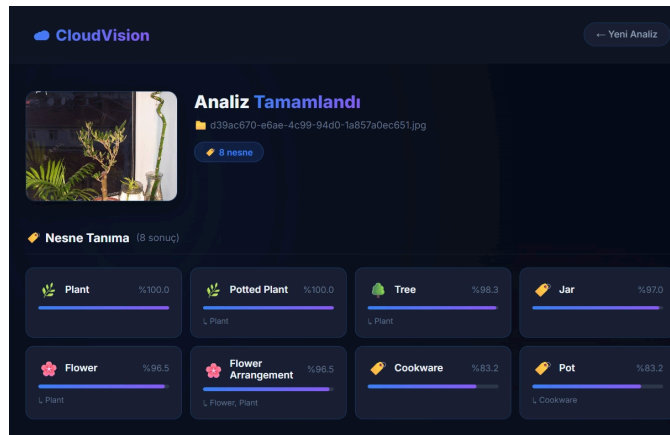
```
reader = imageio.get_reader(tmp_path)
total_frames = reader.count_frames()

# 5 eşit aralıklı kare seç
step = total_frames // 5
indices = [i * step for i in range(5)]

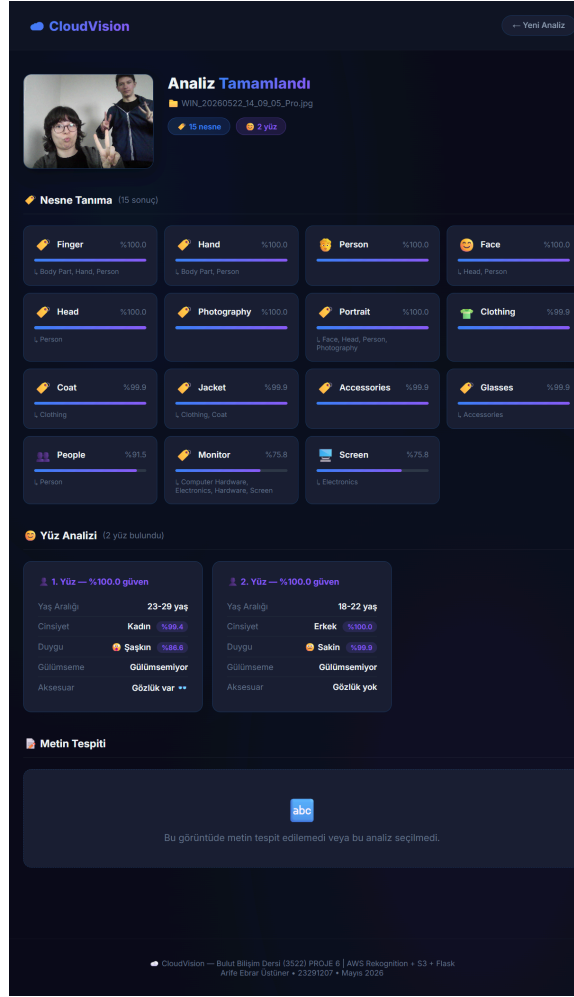
for idx in indices:
    frame_array = reader.get_data(idx) # NumPy array
    pil_image = Image.fromarray(frame_array)
    # JPEG'e çevir → S3'e yükle → Rekognition'a gönder
```

5. Test Sonuçları

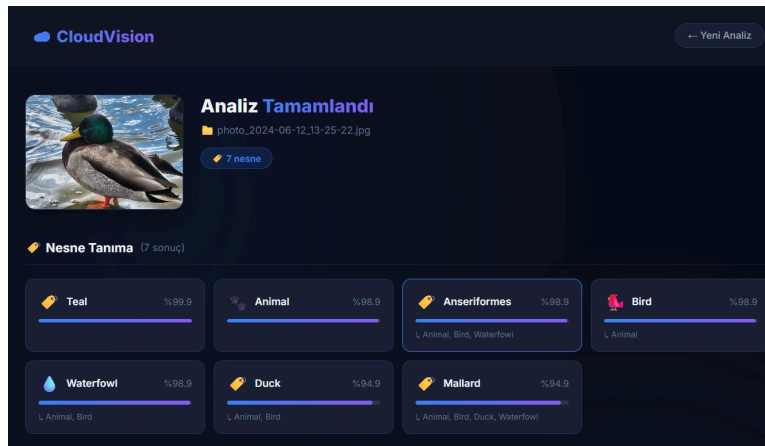
Test 1 — Bitki Fotoğrafı (Nesne Tanıma)



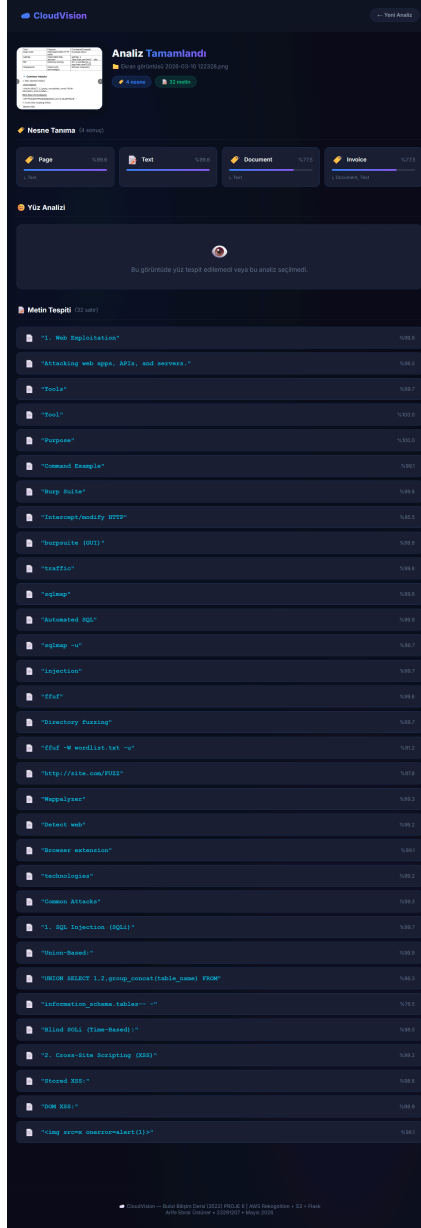
Test 2 — İnsan Fotoğrafı (Yüz Analizi)



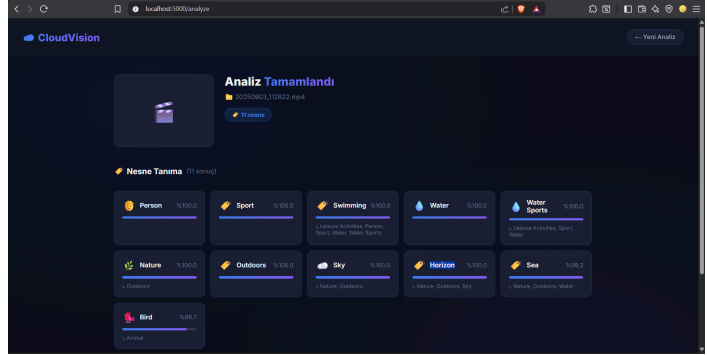
Test 3 — Ördek Fotoğrafı (Detaylı Nesne Tanıma)



Test 4 — Yazılı Belge (Metin Tespiti / OCR)



Test 5 — Video Analizi (Kare Birleştirme)



6. Karşılaşılan Hatalar ve Çözümleri

Hata	Sebebi	Çözüm
AccessDenied: s3:PutObject	IAM kullanıcısının S3 yazma izni yoktu	AmazonS3FullAccess politikası eklendi
AccessDenied: Rekognition	Rekognition izni eksikti	AmazonRekognitionFullAccess

Hata	Sebebe	Çözüm
		eklendi
<code>Request Entity Too Large</code>	Video 50MB limitini aşıyordu	Flask limiti 200MB'a çıkarıldı
<code>Unsupported codec/format</code>	Rekognition Video API, telefon codec'ini (HEVC) reddetti	Video API yerine kare çıkarma yöntemi kullanıldı
<code>ffmpeg plugin not found</code>	imageio v3 plugin ismi uyumsuzluğu	<code>imageio.get_reader()</code> (eski API) kullanıldı

7. Sonuç ve Değerlendirme

Bu proje ile:

1. **AWS Rekognition** servisinin nesne tanıma, yüz analizi ve OCR yetenekleri başarıyla uygulamaya entegre edildi.
2. **AWS S3** ile bulut tabanlı dosya depolama sağlandı; fotoğraf ve video kareleri S3 üzerinden Rekognition'a aktarıldı.
3. **Python + Flask** ile tam işlevsel bir web uygulaması geliştirildi.
4. **Video analizi** için yaratıcı bir çözüm üretildi: videodan 5 eşit kare çıkarılarak her kare analiz edildi ve sonuçlar birleştirilerek tek bir video analiz raporu oluşturuldu. Bu yöntem, codec uyumsuzluklarını ortadan kaldırmakta ve her türlü video formatını desteklemektedir.
5. IAM üzerinden güvenli kimlik doğrulama ve yetkilendirme yönetimi yapıldı.

Proje, kendi yapay zeka modeli eğitmeye gerek kalmadan AWS'nin hazır **AI/ML servislerini** kullanarak güçlü görüntü ve video işleme çözümleri üretilebileceğini göstermiştir.